

Reference architecture

We have already decomposed our UMS into four microservices, namely: *AddService*, *FindService*, *SearchService* and *JournalService*. These use H2 relational databases for storage and Apache Kafka for asynchronous inter-service collaboration. Now, let's refactor the architecture to achieve the following:

1. Replace H2 with MySQL so that the data is saved persistently and shared across all the service instances.
2. Deploy Kafka cluster.
3. Deploy three instances of *AddService*.
4. Deploy six instances of *FindService* and *SearchService*.
5. Deploy one instance of *JournalService*.

Since we have only developed *AddService* so far, we will cover the first three goals in this part. Figure 1 gives our reference architecture.

Spring Boot and MySQL

The *AddService* is currently using the H2 database. As you know, H2 is an in-memory database engine. The data is lost once the engine is restarted. Such behaviour is not desired in the production. We need a database that is persistent. It can be an RDBMS or a NoSQL database like Mongo, etc. We chose MySQL for this illustration.

Since SpringBoot does not offer the MySQL connector out-of-the-box, we need to add it as a dependency in the *pom.xml* of the *AddService*.

```
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
  <scope>runtime</scope>
</dependency>
```

We also need to update *application.properties* to specify the JDBC driver along with a connection string and access details for the MySQL database engine.

```
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.datasource.url=jdbc:mysql://mysqlpdb:3306/glarimy?allow
PublicKeyRetrieval=true&useSSL=false
spring.datasource.username=root
spring.datasource.password=admin
```

Because of the above configuration, the repository of *AddService* attempts to connect to the *glarimy* database on a machine named *mysqlpdb* on port number 3306. We are recording the password in clear text in this configuration, only for simplicity. We will find a better way later!

A few other JPA-specific configurations may also be provided as needed. For example, the following will direct the Hibernate system to scan the code for JPA annotations

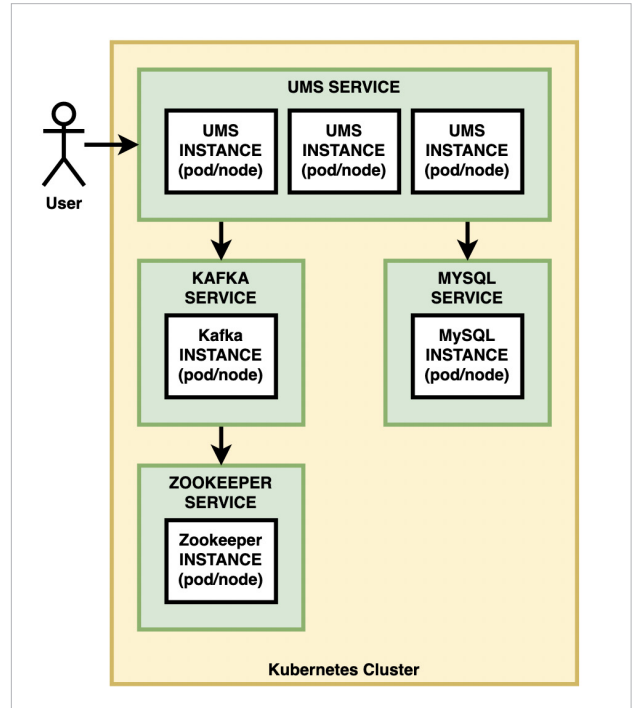


Figure 1: Reference architecture

and keep the schema on the database updated at the time of bootstrapping:

```
spring.jpa.hibernate.ddl-auto=update
spring.jpa.properties.hibernate.dialect=org.hibernate.
dialect.MySQL5Dialect
```

MySQL as a Docker container

Since the *AddService* depends on MySQL, we can update the existing *docker-compose.yml* for deploying and linking it:

```
mysqlpdb:
  image: mysql:latest
  networks:
    - glarimy
  environment:
    - MYSQL_ROOT_PASSWORD=admin
    - MYSQL_DATABASE=glarimy
  volumes:
    - "mysql_data:/glarimy"
```

The above manifest pulls *mysql:latest* image from the Docker Hub and runs the container. The name of the container must be *mysqlpdb* as the *AddService* is looking for the database engine on a machine named *mysqlpdb*. Also, both must run on the same network to resolve the name. Since the *AddService* was configured to run on *glarimy* network (in the previous part), *mysqlpdb* is also configured to run on the same network.